



Université du Québec
à Chicoutimi

Département d'informatique et de mathématique
8INF874 — Cryptographie

Chiffrement homomorphe basé sur LWE

Implémentation logicielle et analyse d'impact de la gestion
de l'erreur

Professeur :
Florentin Thullier

Par :
Justin Bossard
BOSJ08070300

Samuel Plet
PLES07110100

Trimestre d'été
22 juin 2026

Table des matières

1. Introduction	2
1.1. Contexte technologique	2
1.2. Contexte cryptographique	2
1.3. Problématique	2
1.4. Objectifs	2
2. Fondements Théoriques (Chiffrement Homomorphe et LWE)	2
2.1. Le Chiffrement Homomorphe	2
2.2. Le problème LWE et Ring-LWE	3
2.3. L'Anatomie du Bruit Cryptographique	3
2.4. Comparaison des schémas	3
3. Architecture Logicielle et Ingénierie des Paramètres	3
3.1. Modèle d'Architecture	3
3.2. Ingénierie des Paramètres	4
3.3. Algèbre des Opérations Homomorphes	4
3.4. Pile Technologique	4
4. Analyse Expérimentale et Métriques	4
4.1. Protocole d'Évaluation	4
4.2. Suivi de la Dégradation du Bruit	5
4.3. Profilage de la Latence	5
4.4. Empreinte Mémoire et Réseau	5
4.5. Limites de l'implémentation	5
5. Conclusion et Perspectives	6
5.1. Bilan de la preuve de concept	6
5.2. Le concept de Bootstrapping	6
5.3. Évolutions matérielles et industrielles	6
6. Annexes	7
6.1. Backend C++	7
6.2. Serveur python	9
Références	14

1. Introduction

1.1. Contexte technologique

L'externalisation des calculs vers l'informatique en nuage (Cloud) pose un défi majeur de sécurité. Actuellement, les données sont efficacement protégées au repos et en transit par les méthodes de chiffrement standards. Cependant, elles doivent être déchiffrées pour être manipulées par un processeur. Cette étape expose les informations sensibles sur les serveurs distants. Le chiffrement homomorphe (FHE) résout ce problème en permettant d'exécuter des opérations directement sur des données chiffrées, assurant une confidentialité totale en cours d'utilisation.

1.2. Contexte cryptographique

L'émergence de l'ordinateur quantique menace la pérennité des systèmes cryptographiques asymétriques actuels, tels que RSA ou ECC. L'algorithme de Shor rend ces standards vulnérables à court terme. Une transition vers la cryptographie post-quantique (PQC) est donc indispensable. Les algorithmes basés sur la difficulté des problèmes de réseaux euclidiens constituent la principale alternative (Bhoi et al., 2025). Parmi eux, le problème LWE (Learning With Errors) offre une sécurité prouvée contre les attaques classiques et quantiques.

1.3. Problématique

Le fonctionnement du problème LWE repose sur l'injection volontaire d'une erreur mathématique, appelée bruit. Ce bruit est fondamental : il garantit la sécurité sémantique du chiffrement. Paradoxalement, ce même bruit est destructeur pour les calculs. Chaque opération homomorphe (particulièrement la multiplication) amplifie cette erreur. Si le bruit dépasse une certaine limite, le message devient corrompu et le déchiffrement échoue.

De plus, la littérature théorique estime souvent la croissance de cette erreur en se basant sur des modèles de cas moyen. La réalité des circuits arithmétiques complexes montre parfois des accumulations d'erreurs beaucoup plus rapides dues à des distributions à queue lourde (Gao & Zheng, 2025). Il existe donc un écart significatif entre les prédictions théoriques et la dégradation pratique.

1.4. Objectifs

Ce projet vise à évaluer concrètement l'accumulation du bruit dans un système basé sur LWE. L'objectif principal est de concevoir une implémentation logicielle pour mesurer empiriquement la consommation du budget de bruit lors d'opérations successives. Cette preuve de concept permettra d'observer directement le point de rupture cryptographique et d'analyser la latence induite par la gestion de cette erreur.

2. Fondements Théoriques (Chiffrement Homomorphe et LWE)

2.1. Le Chiffrement Homomorphe

Le chiffrement homomorphe permet d'effectuer des calculs mathématiques directement sur des données chiffrées. Un schéma E est dit homomorphe s'il préserve la structure algébrique des données. Pour une opération $\star$ dans le domaine chiffré et $\circ$ dans le domaine clair, l'égalité suivante s'applique :

$$E(m_1) \star E(m_2) = E(m_1 \circ m_2)$$

Ces schémas supportent l'addition et la multiplication. Leur évolution historique se divise en trois catégories :

- **PHE (Partially Homomorphic Encryption)** : Supporte une seule opération mathématique.
- **SWHE (Somewhat Homomorphic Encryption)** : Supporte l'addition et la multiplication, mais pour un nombre limité d'opérations successives (circuit de profondeur finie).

- **FHE (Fully Homomorphic Encryption)** : Supporte une profondeur de calcul infinie grâce au mécanisme de rafraîchissement du bruit (**bootstrapping**).

2.2. Le problème LWE et Ring-LWE

Le chiffrement s'appuie sur le problème mathématique LWE (Learning With Errors). Il consiste à résoudre des systèmes d'équations linéaires volontairement bruités. Ce problème de réseaux euclidiens offre une sécurité robuste face aux ordinateurs quantiques.

Pour des raisons d'efficacité calculatoire, l'implémentation utilise la variante structurée Ring-LWE (RLWE). L'arithmétique n'opère plus sur des matrices, mais sur des polynômes. Cette structure algébrique accélère significativement les opérations. Ces calculs s'effectuent dans un anneau quotient $\frac{\mathbb{Z}_q[X]}{X^{n+1}}$, ce qui empêche l'explosion de la taille des données en réduisant continuellement le degré des polynômes.

La génération des clés repose sur l'équation suivante :

$$b(x) = -a(x) \cdot s(x) + e(x) \bmod q$$

Le polynôme $s(x)$ est la clé secrète. Le polynôme $a(x)$ est un masque aléatoire public. La clé publique est formée par la paire $(b(x), a(x))$.

2.3. L'Anatomie du Bruit Cryptographique

Le terme $e(x)$ représente le bruit cryptographique. Cette erreur probabiliste est strictement nécessaire pour la sécurité sémantique (IND-CPA), mais exige une gestion rigoureuse (Bai et al., 2024).

Cependant, ce bruit entrave le calcul. Chaque évaluation homomorphe amplifie l'erreur sous-jacente. L'addition engendre une croissance linéaire du bruit. La multiplication provoque une croissance exponentielle.

Le texte chiffré dispose d'un budget de bruit limité. L'Effet Falaise (**Cliff Effect**) survient lorsque ce budget est épuisé. Le bruit accumulé déborde alors sur les bits de poids fort réservés au message. À ce stade, la donnée est irrémédiablement corrompue et le déchiffrement échoue.

2.4. Comparaison des schémas

L'implémentation nécessite le choix d'un schéma cryptographique adapté, un processus facilité par la comparaison des propriétés des différents standards (Doan et al., 2023).

Le schéma CKKS utilise une arithmétique à virgule flottante approximative, où le bruit se confond volontairement avec l'erreur de précision numérique (Kim et al., 2020).

Le schéma BFV utilise une arithmétique entière exacte. Le bruit LWE y reste mathématiquement isolé du message clair jusqu'au point de saturation.

Ce projet retient le schéma BFV. L'arithmétique exacte garantit une intégrité absolue tant que le budget de bruit est positif, permettant d'observer rigoureusement le franchissement de la limite d'erreur.

3. Architecture Logicielle et Ingénierie des Paramètres

3.1. Modèle d'Architecture

L'architecture logicielle repose sur une séparation asymétrique stricte. Le client gère la génération des clés, le chiffrement initial et le déchiffrement final. Le serveur, qualifié d'aveugle, reçoit uniquement les textes chiffrés et les clés d'évaluation publiques. Il exécute les opérations mathématiques sans jamais accéder aux données en clair ni à la clé secrète.

3.2. Ingénierie des Paramètres

Le degré du polynôme est fixé à $N = 8192$. Cette valeur définit la dimension du treillis euclidien, offrant un compromis optimal entre le niveau de sécurité et les performances de calcul. L'espace du message en clair (**Plain Modulus**) est dimensionné à $t = 45$ bits. Cette taille élevée prévient les dépassements arithmétiques (**overflows**) et isole spécifiquement l'erreur cryptographique.

L'implémentation exploite l'optimisation SIMD (**Single Instruction, Multiple Data**), également appelée **Batching**. Cette technique s'appuie sur le théorème des restes chinois pour partitionner un unique polynôme en de multiples sous-espaces indépendants. Cela permet d'encoder les 6400 pixels de l'image de test au sein des 8192 emplacements du polynôme.

3.3. Algèbre des Opérations Homomorphes

L'addition homomorphe est employée pour simuler l'ajustement de la luminosité des pixels. Cette opération est mathématiquement linéaire et n'induit qu'une croissance négligeable du bruit.

La multiplication homomorphe est utilisée pour évaluer une fonction de correction Gamma. Cette opération engendre une croissance exponentielle du bruit et génère un polynôme de taille étendue. Une étape de relinéarisation, exploitant les clés d'évaluation (**RelinKeys**), est obligatoirement appliquée après chaque multiplication pour limiter l'expansion mémoire.

3.4. Pile Technologique

L'exécution arithmétique de bas niveau est confiée à la bibliothèque Microsoft SEAL, implémentée en C++17. Ce choix garantit des performances optimales et une latence minimisée par rapport à d'autres bibliothèques pour ce type de calculs (Dhiman et al., 2023; Zhu et al., 2023).

L'orchestration du système s'appuie sur un serveur web développé en Python avec le framework Flask. Le backend Python interagit avec l'exécutable C++ via des appels de sous-processus, collectant les métriques de performance et l'état du bruit au format JSON.

4. Analyse Expérimentale et Métriques

4.1. Protocole d'Évaluation

Le protocole d'évaluation repose sur un cas d'usage de traitement d'image délégué. Le client chiffre une image source en niveaux de gris (80×80 pixels). Le serveur aveugle applique deux transformations homomorphes distinctes : un ajustement de la luminosité et une fonction de correction Gamma. Les résultats déchiffrés sont ensuite comparés mathématiquement à l'image attendue.

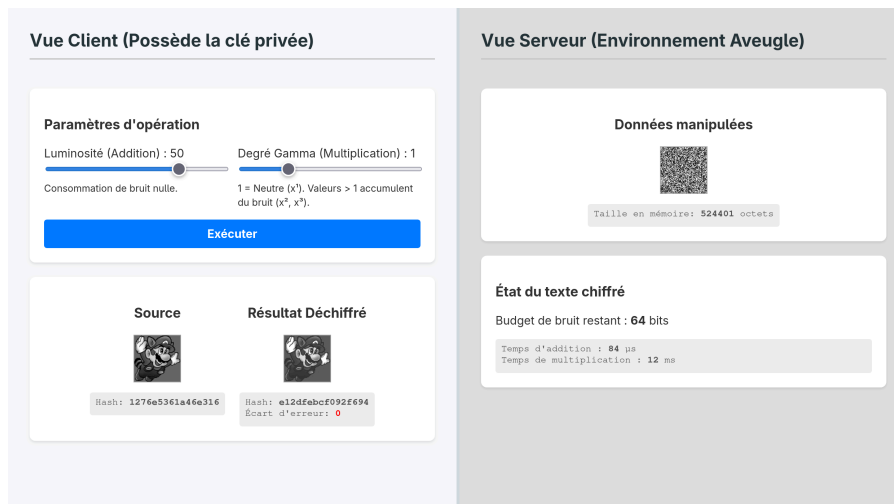


Fig. 1. – Interface de la preuve de concept illustrant l'architecture asymétrique Client/Serveur.

4.2. Suivi de la Dégradation du Bruit

L'implémentation mesure dynamiquement le budget de bruit invariant restant. Avec les paramètres sélectionnés ($N = 8192$, $t = 45$ bits), le budget initial s'élève à environ 135 bits (assurant un niveau de sécurité post-quantique standard de 128 bits).

L'opération d'addition homomorphe ne consomme qu'une fraction négligeable de ce budget (≈ 1 bit). À l'inverse, chaque multiplication homomorphe ampute massivement ce budget de manière constante (environ 32 bits par itération). La profondeur multiplicative maximale du circuit est donc strictement bornée. La décroissance du budget y est linéaire en fonction du nombre d'opérations ($135 \rightarrow 103 \rightarrow 71 \rightarrow 39 \rightarrow 7 \rightarrow 0$ bits).

L'expérimentation valide l'Effet Falaise (**Cliff Effect**). Tant que le budget reste strictement positif, l'écart d'erreur demeure absolument nul. Dès que le budget atteint 0 bit, les données subissent une corruption totale et instantanée.

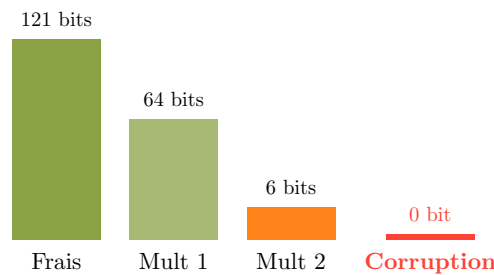


Fig. 2. – Évolution de la consommation du budget de bruit et illustration visuelle de la corruption (Effet Falaise)

4.3. Profilage de la Latence

L'analyse des temps d'exécution, cohérente avec les observations menées lors d'évaluations comparatives de schémas FHE (Jiang & Ju, 2022), révèle une forte asymétrie computationnelle.

L'addition homomorphe s'exécute en moyenne en $130\mu s$. La multiplication homomorphe, combinée à son étape obligatoire de relinéarisation, requiert environ $20ms$. L'opération de mise au carré (optimisée mathématiquement) requiert un temps d'exécution légèrement inférieur à la multiplication standard, mais consomme un budget de bruit strictement identique. Cette asymétrie confirme que la profondeur multiplicative d'un circuit définit son goulot d'étranglement principal.

4.4. Empreinte Mémoire et Réseau

Le chiffrement basé sur LWE induit une expansion massive des données traitées. Un entier standard en clair de 8 octets est transformé en un doublet de polynômes chiffrés pesant approximativement 100 Ko (ratio $\approx 10^4$). L'utilisation de l'optimisation SIMD permet d'amortir partiellement ce ratio.

Cependant, l'impact réseau reste significatif. Le client doit transmettre au serveur les clés d'évaluation publiques (**RelinKeys**). Pour les paramètres actuels, ces clés représentent un volume de données d'environ 16 Mo, pénalisant la bande passante avant le début des calculs.

4.5. Limites de l'implémentation

Bien que fonctionnelle, cette preuve de concept présente plusieurs limites d'ingénierie et d'analyse qui restreignent sa portée théorique :

- **Distribution statistique de l'erreur** : L'outil extrait uniquement la valeur scalaire du budget de bruit (`invariant_noise_budget`). Il ne permet pas d'analyser la distribution exacte des coefficients pour observer empiriquement les phénomènes à queue lourde décrits dans la littérature récente (Gao & Zheng, 2025).

- **Biais méthodologique d'épuisement (Débordement vs Bruit) :** L'évaluation de la fonction Gamma (x^γ) fait croître exponentiellement la valeur mathématique du message. Cela crée un risque de confondre l'épuisement du budget de bruit avec un débordement arithmétique (**overflow**) du module en clair t . Une mesure rigoureuse isolant strictement l'effet du bruit exigerait des multiplications successives par une constante mathématique neutre (valeur 1).
- **Optimisation de la profondeur multiplicative :** La fonction de correction Gamma utilise une boucle d'itération séquentielle. Une approche optimale requerrait un arbre d'exponentiation binaire ou l'algorithme de Paterson-Stockmeyer pour minimiser la profondeur du circuit.

5. Conclusion et Perspectives

5.1. Bilan de la preuve de concept

La preuve de concept valide la viabilité théorique du chiffrement homomorphe basé sur LWE. L'exécution de calculs à l'aveugle préserve strictement l'intégrité des données, conditionnée par le maintien d'un budget de bruit positif. Cependant, des obstacles pratiques majeurs limitent son déploiement immédiat. L'expansion massive de la mémoire et l'accumulation inexorable de l'erreur restreignent cette approche à des circuits de profondeur finie (Levelled-FHE).

5.2. Le concept de Bootstrapping

Le **Bootstrapping** constitue la solution théorique à l'épuisement du bruit en restaurant le budget d'erreur sans exposer le message en clair. Ce mécanisme permet d'atteindre une profondeur de calcul infinie (Li et al., 2026). Toutefois, son coût computationnel est actuellement prohibitif. Son intégration a été exclue de ce projet afin de privilégier l'observation directe de la dégradation de l'erreur, bien que de nouvelles stratégies basées sur l'apprentissage par renforcement émergent pour optimiser son déclenchement (Zhang et al., 2026).

5.3. Évolutions matérielles et industrielles

La démocratisation du chiffrement homomorphe exige une accélération matérielle spécifique. La multiplication polynomiale sature la bande passante mémoire des processeurs classiques. La conception de circuits dédiés (FPGA ou ASIC) est indispensable pour assurer la parallélisation massive requise.

Malgré ces défis, les perspectives d'intégration industrielle se concrétisent. Un cas d'usage pertinent est la recherche visuelle chiffrée sur appareil mobile. Dans ce modèle, un client soumet une image chiffrée à un serveur. Ce dernier évalue homomorphiquement la requête contre une base de données mondiale et retourne un résultat chiffré, garantissant ainsi une confidentialité absolue.

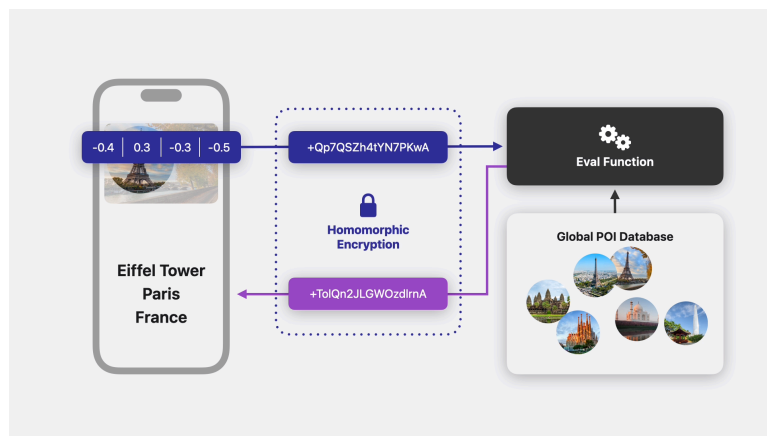


Fig. 3. – Architecture asymétrique pour la recherche visuelle chiffrée sur appareil mobile. Source : Apple Machine Learning Research (<https://machinelearning.apple.com/research/homomorphic-encryption>)

6. Annexes

6.1. Backend C++

Pour compiler : `cmake -B build && cmake --build build`.

```
#include <iostream>
#include <vector>
#include <string>
#include <cmath>
#include <iomanip>
#include <sstream>
#include <algorithm>
#include <chrono>
#include <seal/seal.h>
#include "stb_image.h"
#include "stb_image_write.h"

using namespace seal;

std::string compute_hash(const std::vector<unsigned char>& data) {
    unsigned long hash = 5381;
    for (unsigned char c : data) hash = ((hash << 5) + hash) + c;
    std::stringstream ss;
    ss << std::hex << hash;
    return ss.str();
}

int main(int argc, char* argv[]) {
    if (argc < 6) return 1;

    std::string input_path = argv[1];
    int brightness = std::stoi(argv[2]);
    int gamma_degree = std::stoi(argv[3]);
    std::string output_path = argv[4];
    std::string server_view_path = argv[5];

    if (gamma_degree < 1) gamma_degree = 1;

    int width, height, channels;
    unsigned char *img_data = stbi_load(input_path.c_str(), &width, &height, &channels,
1);
    if (!img_data) return 1;
    size_t pixel_count = width * height;

    std::vector<unsigned char> src_pixels(img_data, img_data + pixel_count);
    std::string src_hash = compute_hash(src_pixels);

    EncryptionParameters parms(scheme_type::bfv);
    size_t poly_modulus_degree = 8192;
    parms.set_poly_modulus_degree(poly_modulus_degree);
    parms.set_coeff_modulus(CoeffModulus::BFVDefault(poly_modulus_degree));
    parms.set_plain_modulus(PlainModulus::Batching(poly_modulus_degree, 45));

    SEALContext context(parms);
    KeyGenerator keygen(context);
    SecretKey secret_key = keygen.secret_key();
    PublicKey public_key;
```



```

keygen.create_public_key(public_key);
RelinKeys relin_keys;
keygen.create_relin_keys(relin_keys);

Encryptor encryptor(context, public_key);
Evaluator evaluator(context);
Decryptor decryptor(context, secret_key);
BatchEncoder batch_encoder(context);

// --- CLIENT : Chiffrement ---
std::vector<uint64_t> image_vector(poly_modulus_degree, 0ULL);
for (size_t i = 0; i < pixel_count; i++) image_vector[i] = img_data[i];

Plaintext plain_image;
batch_encoder.encode(image_vector, plain_image);
Ciphertext encrypted_image;
encryptor.encrypt(plain_image, encrypted_image);

// --- SERVEUR : 1. Évaluation Polynomiale (Correction Gamma) ---
Ciphertext encrypted_result = encrypted_image;
uint64_t scale_factor = static_cast<uint64_t>(std::pow(255.0, gamma_degree - 1));

long long time_mult_ms = 0;
auto start_mult = std::chrono::high_resolution_clock::now();

for (int it = 1; it < gamma_degree; ++it) {
    evaluator.multiply_inplace(encrypted_result, encrypted_image);
    evaluator.relinearize_inplace(encrypted_result, relin_keys);
}

auto end_mult = std::chrono::high_resolution_clock::now();
if (gamma_degree > 1) {
    time_mult_ms = std::chrono::duration_cast<std::chrono::milliseconds>(end_mult -
start_mult).count();
}

// --- SERVEUR : 2. Addition Linéaire (Luminosité avec mise à l'échelle) ---
long long time_add_us = 0;
if (brightness != 0) {
    int64_t scaled_brightness = static_cast<int64_t>(brightness) * scale_factor;
    std::vector<uint64_t> vec_B(poly_modulus_degree, std::abs(scaled_brightness));
    Plaintext plain_B;
    batch_encoder.encode(vec_B, plain_B);

    auto start_add = std::chrono::high_resolution_clock::now();
    if (scaled_brightness > 0) evaluator.add_plain_inplace(encrypted_result, plain_B);
    else evaluator.sub_plain_inplace(encrypted_result, plain_B);
    auto end_add = std::chrono::high_resolution_clock::now();

    time_add_us = std::chrono::duration_cast<std::chrono::microseconds>(end_add -
start_add).count();
}

// Extraction de la vue serveur
std::vector<unsigned char> server_pixels(pixel_count);
const uint64_t* cipher_data = encrypted_result.data(0);
for (size_t i = 0; i < pixel_count; i++) {
    server_pixels[i] = static_cast<unsigned char>(cipher_data[i] % 256);
}

```

```

    }
    stbi_write_png(server_view_path.c_str(), width, height, 1, server_pixels.data(),
width);

    // --- CLIENT : Déchiffrement et Validation ---
    Plaintext plain_result;
    decryptor.decrypt(encrypted_result, plain_result);
    std::vector<uint64_t> decoded_result;
    batch_encoder.decode(plain_result, decoded_result);

    std::vector<unsigned char> final_pixels(pixel_count);
    double total_diff = 0.0;
    uint64_t plain_mod = parms.plain_modulus().value();

    for (size_t i = 0; i < pixel_count; i++) {
        uint64_t val = decoded_result[i];
        int64_t actual_val = val > (plain_mod / 2) ? val - plain_mod : val;

        // Division par le facteur d'échelle
        double pixel_val = static_cast<double>(actual_val) / scale_factor;
        final_pixels[i] = static_cast<unsigned
char>(std::clamp(static_cast<int>(std::round(pixel_val)), 0, 255));

        // Vérité terrain
        double expected_pixel = std::pow(static_cast<double>(img_data[i]), gamma_degree) /
scale_factor + brightness;
        int expected = std::clamp(static_cast<int>(std::round(expected_pixel)), 0, 255);

        total_diff += std::abs(static_cast<int>(final_pixels[i]) - expected);
    }

    stbi_image_free(img_data);
    stbi_write_png(output_path.c_str(), width, height, 1, final_pixels.data(), width);

    std::cout << "{\n"
        << "  \"budget_final\": " <<
decryptor.invariant_noise_budget(encrypted_result) << ",\n"
        << "  \"ecart_moyen\": " << std::fixed << std::setprecision(4) <<
(total_diff / pixel_count) << ",\n"
        << "  \"hash_src\": \"" << src_hash << "\",\n"
        << "  \"hash_final\": \"" << compute_hash(final_pixels) << "\",\n"
        << "  \"taille_octets\": " << encrypted_result.save_size() << ",\n"
        << "  \"temps_mult_ms\": " << time_mult_ms << ",\n"
        << "  \"temps_add_us\": " << time_add_us << "\n"
        << "}\n";

    return 0;
}

```

6.2. Serveur python

Pour lancer le serveur : `uv run server.py`.

```

#!/usr/bin/env python3
# /// script
# dependencies = [
#     "flask",
#     "pillow",
# ]

```

```
# ///
```

```
from flask import Flask, request, jsonify, render_template_string
import subprocess
import json
import os
import time
```

```
app = Flask(__name__)
```

```
HTML_TEMPLATE = """
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <title>Démonstration FHE</title>
    <style>
        body { font-family: system-ui, sans-serif; margin: 0; background: #e0e0e0; color:
#333; }
        .container { display: flex; width: 100vw; min-height: 100vh; }
        .column { flex: 1; padding: 30px; display: flex; flex-direction: column; gap:
20px; }
        .client-side { background: #f8f9fa; border-right: 4px solid #cfd8dc; }
        .server-side { background: #e0e0e0; }
        .card { background: white; padding: 20px; border-radius: 8px; box-shadow: 0 2px
4px rgba(0,0,0,0.1); }
        .img-container { display: flex; gap: 20px; align-items: flex-start; justify-
content: center; }
        .img-box { text-align: center; }
        img { max-width: 100%; image-rendering: pixelated; border: 1px solid #aaa;
background: #fff; }
        .controls { display: grid; grid-template-columns: 1fr 1fr; gap: 15px; }
        button { grid-column: span 2; padding: 10px; background: #007bff; color: white;
border: none; border-radius: 4px; cursor: pointer; font-size: 16px; }
        button:hover { background: #0056b3; }
        .metric { font-family: monospace; font-size: 13px; color: #555; background: #eee;
padding: 8px; border-radius: 4px; text-align: left; margin-top: 10px;}
        h2 { margin-top: 0; color: #263238; border-bottom: 2px solid #ccc; padding-bottom:
10px; }
    </style>
</head>
<body>
    <div class="container">

        <div class="column client-side">
            <h2>Vue Client (Possède la clé privée)</h2>

            <div class="card">
                <h3>Paramètres d'opération</h3>
                <div class="controls">
                    <div>
                        <label>Luminosité (Addition) : <span id="val_b">0</span></
label><br>
                        <input type="range" id="brightness" min="-100" max="100" value="0"
oninput="document.getElementById('val_b').innerText = this.value" style="width:100%">
                        <small>Consommation de bruit nulle.</small>
                    </div>
                </div>
            </div>
        </div>
    </div>
</body>
</html>
"""
```

```

        <label>Degré Gamma (Multiplication) : <span id="val_c">1</span></
label><br>
        <input type="range" id="gamma_degree" min="1" max="5" value="1"
oninput="document.getElementById('val_c').innerText = this.value" style="width:100%">
        <small>1 = Neutre ( $x^1$ ). Valeurs > 1 accumulent du bruit ( $x^2$ ,
 $x^3$ ).</small>
    </div>
    <button onclick="applyFilter()">Exécuter</button>
</div>
</div>

<div class="card img-container">
    <div class="img-box">
        <h3>Source</h3>
        
        <div class="metric">
            Hash: <strong id="hash_src">-</strong>
        </div>
    </div>
    <div class="img-box">
        <h3>Résultat Déchiffré</h3>
        <img id="resultImage" src="" alt="En attente...">
        <div class="metric">
            Hash: <strong id="hash_final">-</strong><br>
            Écart d'erreur: <strong id="ecart" style="color:red">-</strong>
        </div>
    </div>
</div>
</div>

<div class="column server-side">
    <h2>Vue Serveur (Environnement Aveugle)</h2>

    <div class="card img-container">
        <div class="img-box">
            <h3>Données manipulées</h3>
            <img id="serverImage" src="" alt="En attente...">
            <div class="metric">
                Taille en mémoire: <strong id="cipher_size">-</strong> octets
            </div>
        </div>
    </div>

    <div class="card">
        <h3>État du texte chiffré</h3>
        <p>Budget de bruit restant : <strong id="budget">-</strong> bits</p>
        <div class="metric">
            Temps d'addition : <strong id="time_add">-</strong>  $\mu$ s<br>
            Temps de multiplication : <strong id="time_mult">-</strong> ms
        </div>
    </div>
</div>

</div>

<script>
    function applyFilter() {
        let b = document.getElementById('brightness').value;

```

```
        let c = document.getElementById('gamma_degree').value;

        fetch('/process', {
            method: 'POST',
            headers: {'Content-Type': 'application/json'},
            body: JSON.stringify({brightness: b, gamma_degree: c})
        })
        .then(r => r.json())
        .then(data => {
            if(data.error) return alert(data.error);

            document.getElementById('resultImage').src = '/' + data.out_img;
            document.getElementById('serverImage').src = '/' + data.srv_img;

            document.getElementById('cipher_size').innerText = data.taille_octets;
            document.getElementById('budget').innerText = data.budget_final;
            document.getElementById('time_add').innerText = data.temps_add_us || 0;
            document.getElementById('time_mult').innerText = data.temps_mult_ms || 0;

            document.getElementById('hash_src').innerText = data.hash_src;
            document.getElementById('hash_final').innerText = data.hash_final;
            document.getElementById('ecart').innerText = data.ecart_moyen;
        });
    }
</script>
</body>
</html>
"""

@app.route('/')
def index():
    return render_template_string(HTML_TEMPLATE)

@app.route('/process', methods=['POST'])
def process():
    data = request.json
    brightness = data.get('brightness', 0)
    gamma_degree = data.get('gamma_degree', 1)

    ts = int(time.time() * 1000)
    out_file = f"static/out_{ts}.png"
    srv_file = f"static/srv_{ts}.png"

    result = subprocess.run(
        [
            "./build/main",
            "static/input.png",
            str(brightness),
            str(gamma_degree),
            out_file,
            srv_file
        ],
        capture_output=True, text=True
    )

    try:
        parsed = json.loads(result.stdout)
        parsed["out_img"] = out_file
```

```
        parsed["srv_img"] = srv_file
        return jsonify(parsed)
    except json.JSONDecodeError:
        return jsonify({"error": "Erreur d'exécution C++", "details": result.stderr}), 500

if __name__ == '__main__':
    os.makedirs("static", exist_ok=True)
    if not os.path.exists("static/input.png"):
        from PIL import Image
        Image.new('L', (80, 80), color=128).save("static/input.png")
    app.run(port=5000, debug=False)
```

Références

- Bai, L., Bai, L., Li, Y., & Li, Z. (2024). Research on Noise Management Technology for Fully Homomorphic Encryption. *IEEE Access*, 12, 135564-135576. <https://doi.org/10.1109/ACCESS.2024.3461729>
- Bhoi, S. S., Arakala, A., Corman, A. B., & Rao, A. (2025). Post-Quantum Homomorphic Encryption: A Case for Code-Based Alternatives. *Cryptography*, 9(2), 31. <https://doi.org/10.3390/cryptography9020031>
- Dhiman, S., Mahato, G. K., & Chakraborty, S. K. (2023). Homomorphic Encryption Library, Framework, Toolkit and Accelerator: A Review. *SN Computer Science*, 5(1), 24. <https://doi.org/10.1007/s42979-023-02316-9>
- Doan, T. V. T., Messai, M.-L., Gavin, G., & Darmont, J. (2023). A survey on implementations of homomorphic encryption schemes. *The Journal of Supercomputing*, 79(13), 15098-15139. <https://doi.org/10.1007/s11227-023-05233-z>
- Gao, M., & Zheng, H. (2025). A Critique on Average-Case Noise Analysis in RLWE-Based Homomorphic Encryption. *Proceedings of the 13th Workshop on Encrypted Computing & Applied Homomorphic Computing, WAHC '25*, 26-37. <https://doi.org/10.1145/3733811.3767312>
- Jiang, L., & Ju, L. (2022, mars 1). *FHEBench: Benchmarking Fully Homomorphic Encryption Schemes*. arXiv. <https://doi.org/10.48550/arXiv.2203.00728>
- Kim, A., Papadimitriou, A., & Polyakov, Y. (2020,). *Approximate Homomorphic Encryption with Reduced Approximation Error*. <https://eprint.iacr.org/2020/1118>
- Li, T., Wang, Z., Zhao, L., Wang, R., Niu, Q., Lu, X., Meng, D., & Hou, R. (2026). A survey of optimization techniques for bootstrapping algorithms in FHE. *Cybersecurity*, 9(1), 150. <https://doi.org/10.1186/s42400-026-00571-w>
- Zhang, C., Bai, F., Wan, J., & Chen, Y. (2026). A Reinforcement Learning-Based Optimization Strategy for Noise Budget Management in Homomorphically Encrypted Deep Network Inference. *Electronics*, 15(2), 275. <https://doi.org/10.3390/electronics15020275>
- Zhu, H., Suzuki, T., & Yamana, H. (2023). Performance Comparison of Homomorphic Encrypted Convolutional Neural Network Inference Among HELib, Microsoft SEAL and OpenFHE. *2023 IEEE Asia-Pacific Conference on Computer Science and Data Engineering (CSDE)*, 1-7. <https://doi.org/10.1109/CSDE59766.2023.10487709>